

MẠNG NƠ-RON HỒI QUY (RNN)

Từ Hạn Chế của MLP & 1D-CNN đến Nguyên Lý Xử Lý Dữ Liệu Chuỗi có Trạng Thái Ẩn

Nhóm thực hiện:

Văn Thị Mai Linh Nguyễn Văn Minh Lực Giáp Minh Hiếu Nguyễn Nam Hải

Chuyên đề Học Sâu (Deep Learning)

*"Feedforward nhìn thế giới qua từng bức ảnh rời rạc;
RNN thấu hiểu dữ liệu qua dòng chảy liên tục của thời gian."*

Bản chất Dữ liệu dạng Chuỗi (Sequential Data)

Thứ tự xuất hiện quyết định hoàn toàn ngữ nghĩa của dữ liệu

- **Đặc tính trật tự:** Mỗi điểm dữ liệu tại thời điểm t luôn mang mối tương quan mật thiết với các điểm quá khứ $t - 1, t - 2, \dots$
- **Hai dạng chính:** Dữ liệu token rời rạc (Text, DNA) và dữ liệu chuỗi số liên tục (Giá vàng, cổ phiếu, nhịp tim).

Chuỗi Rời rạc (NLP / Văn bản)

”Hà Nội không mưa” $\neq$ ”Mưa không Hà Nội”

→ Câu đầu có nghĩa rõ ràng, câu sau vô nghĩa hoặc biến đổi ngữ nghĩa.

Chuỗi Liên tục (Time-Series)

Giá vàng: $[82.5 \rightarrow 83.1 \rightarrow 84.0 \rightarrow 85.2]$ (Tăng)

Giá vàng: $[85.2 \rightarrow 84.0 \rightarrow 83.1 \rightarrow 82.5]$ (Giảm)

Bài toán Dự đoán Phần tử Kế tiếp (Next Step Prediction)

Mục tiêu: Cho chuỗi quan sát quá khứ $X = (x_1, x_2, \dots, x_t)$, dự đoán giá trị tương lai x_{t+1} .

Bài toán 1: Next Word Prediction (NLP)

Ngữ cảnh: "We" $\rightarrow$ "are" $\rightarrow$ "learning" $\rightarrow$ "AI" $\rightarrow$ [?]
 $\rightarrow$ Target kỳ vọng: "is"

Bài toán 2: Dự báo Chuỗi thời gian (Time-Series)

Cửa sổ trượt (30 ngày lịch sử):
 $[P(t-29), P(t-28), \dots, P(t-1), P(t)] \rightarrow [P(t+1)]$
 $\rightarrow$ Dự báo biến động ngày 31.

Hai Pipeline Tiền xử lý Đặc trưng

Dữ liệu thô không thể đưa trực tiếp vào mạng mà phải đi qua pipeline chuẩn hóa:

Văn bản (Text Pipeline)

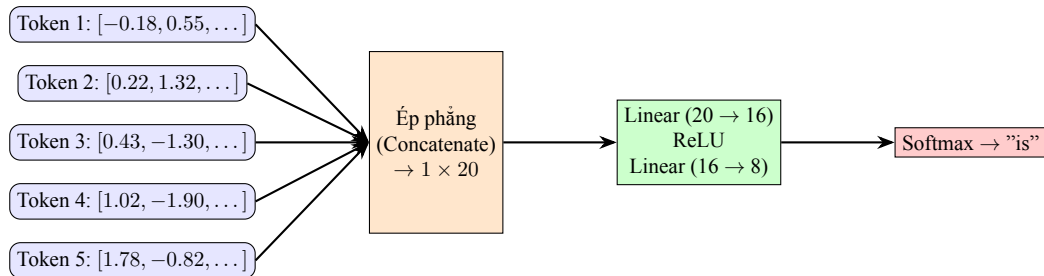
- 1 Văn bản thô: "giá vàng tăng"
- 2 Tokenizer: ["giá", "vàng", "tăng"]
- 3 Vocab Index: [2, 3, 4]
- 4 Padding: [2, 3, 4, 0, 0]
- 5 Embedding: Tensor [Batch, Seq_Len, d]

Chuỗi số (Time-Series Pipeline)

- 1 File CSV thô: Date, Price
- 2 Làm sạch & Chia tập: Train 70%, Val 15%, Test 15%
- 3 Scaling: $\log_{1p}(\text{Price}) \rightarrow \text{MinMax}$ (chỉ fit trên Train)
- 4 Windowing: Cửa sổ trượt 30 ngày
- 5 Đầu vào: Tensor [N, 30, 1]

Tiếp cận bằng MLP & Cơ chế Ghép phẳng (Flatten)

Mạng truyền thẳng (MLP) bắt buộc phải biến toàn bộ ngữ cảnh thành một vector phẳng duy nhất.



Ví dụ: Tokenizer & Vocabulary cho Next-Word Prediction

1. Code Python: tokenize, chuẩn hóa và tra ID

```
text = "We are learning AI"
tokens = text.split()
tokens = [tok.lower() for tok in tokens]

vocab = {
    "[UNK]": 0, "[PAD]": 1, "ai": 2,
    "a": 3, "are": 4, "cs": 5,
    "is": 6, "learning": 7,
}

ids = [vocab.get(tok, vocab["[UNK]"]) for tok in tokens]
target = vocab["is"]
```

Index (ID)	Word / Token
0	[UNK] (We)
1	[PAD]
2	ai
3	a
4	are
5	cs
6	is (Target)
7	learning

Tokenizer chỉ biến chuỗi thành token; vocabulary biến token thành ID.

2. Kết quả đầu ra

```
tokens = ["we", "are", "learning", "ai"]
ids = [0, 4, 7, 2], target = 6
```

Padding & Embedding: biến ID thành vector số

Code PyTorch: padding và embedding

```
import torch
import torch.nn as nn
import torch.nn.functional as F

ids = torch.tensor([0, 4, 7, 2])
ids = F.pad(ids, (0, 1), value=1)
embedding = nn.Embedding(8, 4)
X = embedding(ids)
```

Kết quả về kích thước

```
ids = [0, 4, 7, 2, 1]
X.shape = torch.Size([5, 4])
```

Ý nghĩa của từng bước

$L = 5$ là cửa sổ đầu vào cố định; [PAD] chỉ được thêm để mọi mẫu có cùng chiều dài.

Nhãn cần dự đoán:

$$y = 6 \iff \text{is}$$

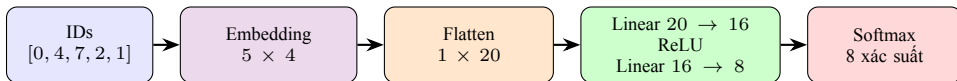
Embedding lookup

Mã trận embedding $E \in \mathbb{R}^{|V| \times d} = \mathbb{R}^{8 \times 4}$.

Mỗi ID được thay bằng một vector học được:

$$\begin{bmatrix} 0 \\ 4 \\ 7 \\ 2 \\ 1 \end{bmatrix} \longrightarrow \begin{bmatrix} e_0 \\ e_4 \\ e_7 \\ e_2 \\ e_1 \end{bmatrix} \in \mathbb{R}^{5 \times 4}$$

MLP dự đoán từ tiếp theo: Forward Pass



Code PyTorch: Flatten và MLP

```
import torch
import torch.nn as nn

X = torch.randn(5, 4)          # output của Embedding
mlp = nn.Sequential(
    nn.Linear(5 * 4, 16), nn.ReLU(),
    nn.Linear(16, 8)
)
x = X.flatten().unsqueeze(0)    # [1, 20]
logits = mlp(x)                 # [1, 8]
```

Kích thước tensor

```
X.shape = [5, 4]
x.shape = [1, 20]
logits.shape = [1, 8]
```

Flatten biến 5 vector embedding thành một vector phẳng; MLP không còn thấy trạng thái theo thời gian.

Từ Softmax đến từ được dự đoán: Loss

Code PyTorch: Softmax và Cross-Entropy

```
import torch
import torch.nn.functional as F
target_probs = torch.tensor([
    [.02, .04, .05, .02, .08, .06, .62, .11]])
logits = target_probs.log()
probs = F.softmax(logits, dim=-1)
y = torch.tensor([6])
loss = F.cross_entropy(logits, y)
print(probs.argmax(dim=-1).item()) # 6 = is
print(loss.item())                 # -log(0.62)
```

Ví dụ đầu ra

Token	Xác suất
ai	0.05
are	0.08
is	0.62
learning	0.11
Các token khác	0.14

$$\hat{y} = \arg \max_i p_i = 6$$

Dự đoán: “is”

Ý nghĩa

$p_i = \text{softmax}(z)_i$, nhân đúng là $y = 6$ (từ is):

$$\mathcal{L} = -\log p_6$$

Muốn loss nhỏ, mô hình phải đẩy xác suất của is lên cao.

Từ giới hạn của MLP đến nhu cầu về RNN

MLP: ĐIỂM NGHẼN

1. Cửa sổ cố định L

Câu dài bị cắt; câu ngắn phải thêm padding.

2. Flatten rời ngữ cảnh

Vị trí 1 và 5 là hai tọa độ độc lập; không có h_{t-1} .

3. Tham số tăng theo L

Flatten: Ld đầu vào $\rightarrow H$ neuron.

$W \in \mathbb{R}^{H \times Ld}$: $H(Ld)$ trọng số.

$b \in \mathbb{R}^H$: H bias $\Rightarrow H(Ld) + H$.

$64(30 \cdot 20) + 64 = 38\,464$ tham số.

bộ nhớ linh hoạt

giữ thứ tự

chia sẻ

RNN: CƠ CHẾ ĐÁP ỨNG

Trạng thái ẩn h_t

Đọc từng bước, giữ ngữ cảnh; nhận chuỗi dài ngắn khác nhau.

Xử lý theo thứ tự thời gian

$h_t = f(x_t, h_{t-1})$: mang theo ngữ cảnh quá khứ.

Chia sẻ trọng số

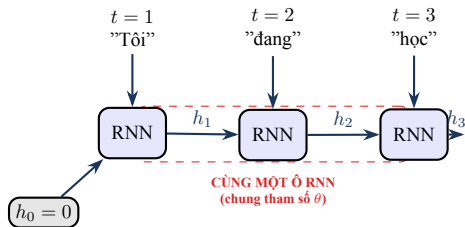
Dùng lại cùng W, U, V ở mọi thời điểm t .

RNN = BỘ NHỚ + THỨ TỰ + CHIA SẺ TRỌNG SỐ

$x_t, h_{t-1} \longrightarrow h_t \longrightarrow y_t$

Ý tưởng Cốt lõi của RNN: Trạng thái Ẩn (h_t)

Thực tế: Con người không bắt đầu tư duy lại từ đầu ở từng chữ, mà liên tục duy trì một dòng ý thức **nén dần thông tin** quá khứ.



h_t = "trí nhớ" nén toàn bộ quá khứ:

- h_1 : chỉ nhớ "Tôi"
- h_2 : nhớ "Tôi" + "đang"
- h_3 : nhớ trọn câu "Tôi đang học"

Phương trình đệ quy của RNN

Mỗi bước chỉ cần **trạng thái liền trước** h_{t-1} và **đầu vào hiện tại** x_t :

$$h_t = f_{\theta}(h_{t-1}, x_t)$$

- **Tóm tắt lịch sử vô hạn:** h_{t-1} đã chứa cả chuỗi quá khứ $x_1, \dots, x_{t-1}$, nên quá khứ dài bao nhiêu cũng được nén vào vector **cố định** d_h chiều.
- **Cập nhật từng bước $O(1)$:** mỗi token đến chỉ cần **một** phép cập nhật; xử lý kiểu streaming, không phải lưu toàn chuỗi trong bộ nhớ.
- **Khác MLP của số trượt:** MLP phải thấy đủ L phần tử trong cửa sổ; RNN duy trì "tóm tắt động" h_t **không phụ thuộc độ dài** chuỗi.

Công thức Toán học – Quá trình Forward

Ba phương trình forward tại bước t , với $x_t \in \mathbb{R}^{d_x}$ (đầu vào), $h_t \in \mathbb{R}^{d_h}$ (trạng thái ẩn), $\hat{y}_t \in \mathbb{R}^{d_y}$ (dự đoán):

$$a_t = \mathbf{W}x_t + \mathbf{U}h_{t-1} + b_h$$

$$h_t = \tanh(a_t)$$

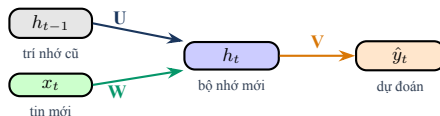
$$\hat{y}_t = \mathbf{V}h_t + b_y$$

tổng nhập: ghép tin mới với trí nhớ cũ

trạng thái ẩn mới, truyền sang bước $t + 1$

đọc bộ nhớ h_t để đưa ra dự đoán

Tham số	Kích thước	Kết nối	Vai trò
$\mathbf{W}$	$d_h \times d_x$	Vào $\rightarrow$ Ẩn	Đọc tin mới x_t
$\mathbf{U}$	$d_h \times d_h$	Ẩn $\rightarrow$ Ẩn	Giữ trí nhớ h_{t-1}
$\mathbf{V}$	$d_y \times d_h$	Ẩn $\rightarrow$ Ra	Đọc ra kết quả
b_h, b_y	d_h, d_y	Bias	Dịch ngưỡng



- **Vì sao hàm tanh?** Tâm 0, chặn $(-1, 1)$ nên chuẩn của h_t ổn định qua các bước, đạo hàm $1 - \tanh^2(a_t)$ gọn; *sigmoid* lệch dương (trung bình 1/2), đạo hàm ≤ 0.25 làm gradient tắt nhanh; *ReLU* không chặn trên, trạng thái cộng dồn dễ bùng nổ.
- **Tên PyTorch tương ứng:** $\mathbf{W} = \text{weight_ih_10}$, $\mathbf{U} = \text{weight_hh_10}$ (Keras: `kernel`, `recurrent_kernel`); mất mát tổng theo thời gian $L = \sum_{t=1}^T \ell(\hat{y}_t, y_t)$ (MSE cho hồi quy).

Ví dụ Tính Toán Forward Từng Bước (số cụ thể)

$$d_x = 2, d_h = 2, d_y = 1; \quad W = \begin{pmatrix} 0.5 & 0.2 \\ -0.1 & 0.4 \end{pmatrix}, \quad U = \begin{pmatrix} 0.3 & 0.0 \\ 0.2 & 0.5 \end{pmatrix}, \quad V = \begin{pmatrix} 1.0 & -1.0 \end{pmatrix},$$
$$\mathbf{x}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \mathbf{h}_0 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad \mathbf{b}_h = \mathbf{0}, \quad b_y = 0.1$$

Bước	Phép tính	Ý nghĩa	Kết quả ($t = 1$)
1	$W \cdot \mathbf{x}_1$	Đọc tin mới	(+0.500, -0.100)
2	$U \cdot \mathbf{h}_0$	Chưa có ký ức (trang trắng)	(+0.000, +0.000)
3	$\mathbf{h}_1 = \tanh(W \cdot \mathbf{x}_1 + U \cdot \mathbf{h}_0 + \mathbf{b}_h)$	Ký ức mới (vector)	(+0.462, -0.100)
4	$\hat{y}_1 = V \cdot \mathbf{h}_1 + b_y$	Đầu ra (số)	+0.662

Bước $t = 2$: lặp nguyên hệ thống. Thay $\mathbf{x}_2 = (0, 1)$, lấy $\mathbf{h}_1$ vừa tính làm *trí nhớ cũ* (lúc này $U \cdot \mathbf{h}_1 \neq \mathbf{0}$: thành phần hồi quy xuất hiện lần đầu); bộ trọng số (W, U, V) giữ nguyên tuyệt đối, chạy dần đến $t = T$.

$$\text{Shape flow: } \mathbf{x} \in \mathbb{R}^{T \times d_x} \longrightarrow \mathbf{h} \in \mathbb{R}^{T \times d_h} \longrightarrow \hat{\mathbf{y}} \in \mathbb{R}^{T \times d_y}$$

Ghi chú: PyTorch `nn.RNN(2, 2) + nn.Linear(2, 1)` nạp đúng W, U, V , đặt 2 bias của `nn.RNN` về 0 và bias `Linear` = 0.1 thì chạy đúng ví dụ (sai số $< 10^{-6}$).

Nguyên lý Chia sẻ Trọng số (Weight Sharing)

Chuỗi dài 5 bước hay 5.000 bước, RNN đều **dùng chung một bộ ma trận** $\mathbf{W}, \mathbf{U}, \mathbf{V}$ duy nhất:

$$\text{Thời điểm } t = 1: \quad h_1 = \tanh(\mathbf{W} \cdot x_1 + \mathbf{U} \cdot h_0 + b_h) \rightarrow y_1 = \mathbf{V} \cdot h_1$$

$$\text{Thời điểm } t = 2: \quad h_2 = \tanh(\mathbf{W} \cdot x_2 + \mathbf{U} \cdot h_1 + b_h) \rightarrow y_2 = \mathbf{V} \cdot h_2$$

$\vdots$ áp dụng cùng một bộ $\mathbf{W}, \mathbf{U}, \mathbf{V}$ tới $t = T$

Tổng số tham số P — không chứa T

$$P = \underbrace{d_h \cdot d_x}_{\mathbf{W}} + \underbrace{d_h^2}_{\mathbf{U}} + \underbrace{d_y \cdot d_h}_{\mathbf{V}} + \underbrace{d_h}_{b_h} + \underbrace{d_y}_{b_y}$$

Ví dụ $d_x = 2, d_h = 64, d_y = 1$:

$P = 128 + 4096 + 64 + 64 + 1 = 4353$ tham số cho **mọi độ dài chuỗi**.

So với MLP cửa sổ $L = 30$

- **MLP** ($d_x = 2, 64$ nơ-ron ẩn): trọng số tầng đầu $= 30 \cdot 2 \cdot 64 = 3840$ (+64 bias), chỉ dùng cho **một độ rộng cửa sổ 30**.
- **RNN**: $\mathbf{W} = 2 \cdot 64 = 128$ tham số nhưng **tái sử dụng 30 lần**.

Bốn lợi ích cốt lõi

- **Tham số hằng số theo T** — chuỗi dài bao nhiêu cũng không tăng thêm tham số.
- **Tổng quát hóa**: mẫu hình xuất hiện ở đầu hay cuối chuỗi đều được nhận diện giống nhau.
- **Tự điều chuẩn hóa**: ít tham số hơn trên cùng dữ liệu $\rightarrow$ kháng quá khớp tốt hơn.
- **Analogy CNN**: kernel CNN chia sẻ trọng số theo **KHÔNG GIAN**, RNN chia sẻ theo **THỜI GIAN**.

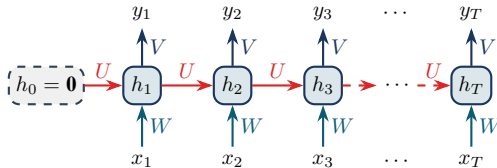
Đồ thị Tính toán dạng Mở cuộn (Unfolded Computation Graph)

Một ô RNN hồi quy khi mở cuộn trở thành một **mạng truyền thẳng cực sâu** có độ sâu đúng bằng T :

DẠNG CUỘN (Folded)



DẠNG MỞ CUỘN (Unfolded)



CÙNG MỘT Ô

— dùng chung W , U , V mọi thời điểm

Mở cuộn = Mạng sâu T tầng

Truyền thẳng **cực sâu**, độ sâu đúng bằng $T \rightarrow$ **BPTT** chính là backprop chạy trên đúng đồ thị này.

U = Đường trí nhớ

Cạnh ngang $h_{t-1} \rightarrow h_t$ (màu đỏ) đưa quá khứ vào hiện tại; **cắt nó** thì mất hết thông tin quá khứ.

T giãn mãi — tham số cố định

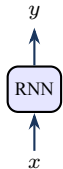
Cùng bộ W, U, V tái sử dụng $\rightarrow$ đồ thị dài thêm bao nhiêu cũng **không thêm trọng số** nào.

Các Dạng Kiến trúc Tương tác Chuỗi (Phần 1)

Tùy số lượng bước vào / bước ra, một ô RNN tạo thành các kiến trúc tương tác chuỗi khác nhau:

One-to-One

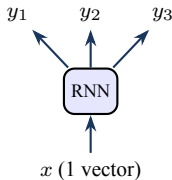
1 bước vào $\rightarrow$ 1 bước ra



Phân loại 1 ảnh — mốc đối chiếu, chưa khai thác ngữ cảnh chuỗi

One-to-Many

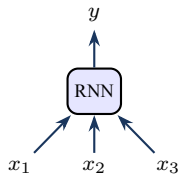
1 bước vào $\rightarrow$ chuỗi ra



Sinh chú thích ảnh (image captioning), sinh nhạc

Many-to-One

chuỗi vào $\rightarrow$ 1 đầu ra

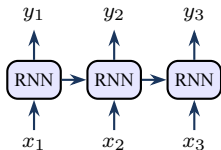


Phân loại cảm xúc review; dự báo giá ngày mai — dạng dùng trong phần thực nghiệm của bài

Các Dạng Kiến trúc Tương tác Chuỗi (Phần 2)

Many-to-Many (Đồng bộ)

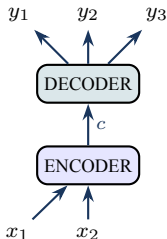
mỗi bước vào $\rightarrow$ 1 nhãn ra



Gán nhãn từ loại POS / NER: mỗi token ra đúng 1 nhãn

Seq2Seq (Encoder-Decoder)

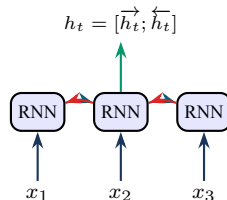
độ dài ra $\neq$ độ dài vào



Encoder nén chuỗi vào thành vector ngữ cảnh c ; Decoder sinh chuỗi ra: dịch máy, tóm tắt

Bidirectional RNN

nhìn cả 2 chiều thời gian

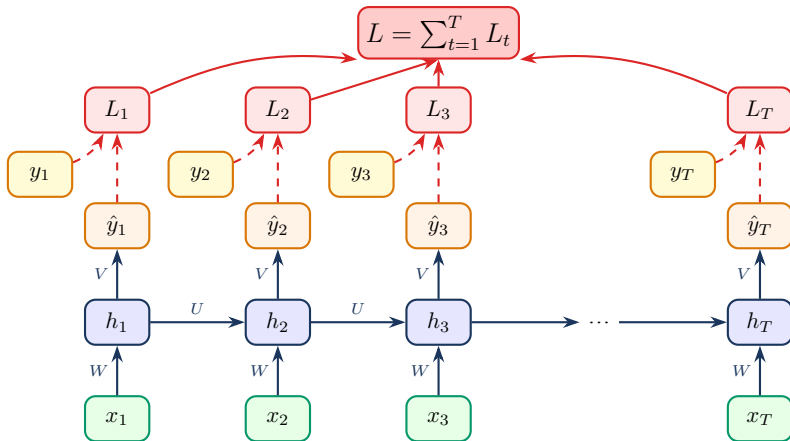


NER, embedding; không dùng dự báo tương lai — nhìn được tương lai = rò rỉ dữ liệu

Liên hệ bài toán asm6

Chuỗi giá quá khứ $\rightarrow$ 1 giá ngày mai $\Rightarrow$ bài toán thực nghiệm thuộc dạng **Many-to-One**.

Hàm mất mát (Loss Function) - Minh họa



Bản chất Hàm mất mát (Loss Function) trong RNN

Khái niệm

Hàm mất mát L (Loss Function) là thước đo độ sai lệch giữa **dự đoán của mô hình** ($\hat{y}$) và **kết quả thực tế** (y). Mục tiêu của quá trình huấn luyện RNN là tìm ra tập trọng số U, W, V sao cho L đạt mức nhỏ nhất.

Đặc trưng Mất mát trong Dữ liệu Chuỗi (Sequential Loss)

Không giống như mạng Feedforward (chỉ đưa ra 1 đầu ra ở lớp cuối), mô hình RNN có thể đưa ra dự đoán $\hat{y}_t$ tại **nhiều bước thời gian** t .

- Hàm mất mát toàn cục L được định nghĩa là **tổng (hoặc trung bình cộng)** của tất cả các hàm mất mát cục bộ L_t tại từng bước thời gian: $L = \sum_{t=1}^T L_t$ (hoặc $L = \frac{1}{T} \sum_{t=1}^T L_t$).
- *Nhắc lại sơ đồ trước:* Tín hiệu sai số L_t từ mọi thời điểm sẽ được thu thập và cộng dồn lại thành L tổng, sau đó mới tiến hành tính đạo hàm lan truyền ngược (BPTT).

Cách chọn Công thức tính Mất mát cục bộ (L_t)

Việc chọn công thức tính L_t hoàn toàn phụ thuộc vào **bản chất của đầu ra** bài toán:

1. Bài toán Hồi quy (Regression)

Dự đoán một giá trị số thực liên tục (VD: Dự báo giá vàng ngày mai, dự báo nhiệt độ).

- **Mean Squared Error (MSE):** Phạt nặng các sai số lớn. Là Loss phổ biến nhất.

$$L_t = \frac{1}{2}(y_t - \hat{y}_t)^2$$

- **Mean Absolute Error (MAE):** Bền vững (robust) hơn với các nhiễu ngoại lai.

$$L_t = |y_t - \hat{y}_t|$$

2. Bài toán Phân loại (Classification)

Dự đoán xác suất thuộc về các nhóm (VD: Phân loại cảm xúc văn bản, Dịch máy).

- **Cross-Entropy Loss (Entropy Chéo):** Đo lường khoảng cách giữa 2 phân phối xác suất.

$$L_t = - \sum_{c=1}^C y_{t,c} \log(\hat{y}_{t,c})$$

Trong đó $y_{t,c}$ là nhãn thật (One-hot vector) và $\hat{y}_{t,c}$ là xác suất dự đoán (qua Softmax).

Backpropagation Through Time (BPTT)

Có 3 tham số ta cần phải tìm là W, U, V . Để thực hiện gradient descent, ta cần tính: $\frac{\partial L}{\partial U}, \frac{\partial L}{\partial V}, \frac{\partial L}{\partial W}$.
Tính đạo hàm với V thì khá đơn giản vì nó chỉ phụ thuộc trực tiếp vào bước hiện tại:

$$\frac{\partial L}{\partial V} = \sum_{t=1}^T \frac{\partial L_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial V}$$

Tuy nhiên với U, W thì lại khác. Xét tổng quát tại thời điểm T :

$$\frac{\partial L}{\partial U} = \sum_{t=1}^T \frac{\partial L_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial h_t} \cdot \frac{\partial h_t}{\partial U}$$

Do trạng thái $h_t = \tanh(U \cdot h_{t-1} + W \cdot x_t)$ có h_{t-1} phụ thuộc vào U , ta phải áp dụng Chain Rule truy hồi liên tục về tận thời điểm đầu tiên.

BPTT: Đạo hàm cho U

Sử dụng công thức đạo hàm hàm hợp $(f(x) \cdot g(x))' = f'(x)g(x) + f(x)g'(x)$, ta có:

$$\frac{\partial h_T}{\partial U} = \frac{\partial h'_T}{\partial U} + \frac{\partial h_T}{\partial h_{T-1}} \cdot \frac{\partial h_{T-1}}{\partial U}$$

trong đó $\frac{\partial h'_T}{\partial U}$ là đạo hàm của h_T với U khi coi h_{T-1} là hằng số đối với U .

Tương tự, trong biểu thức h_{T-1} có h_{T-2} phụ thuộc vào U ...nên áp dụng công thức trên và chuỗi Chain Rule, ta được:

$$\frac{\partial L}{\partial U} = \sum_{t=1}^T \sum_{i=0}^t \frac{\partial L_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial h_t} \cdot \frac{\partial h_t}{\partial h_i} \cdot \frac{\partial h'_i}{\partial U}$$

trong đó $\frac{\partial h_t}{\partial h_i} = \prod_{j=i}^{t-1} \frac{\partial h_{j+1}}{\partial h_j}$ và $\frac{\partial h'_i}{\partial U}$ là đạo hàm của h_i với U khi coi h_{i-1} là hằng số.

BPTT: Đạo hàm cho W

Hoàn toàn tương tự với ma trận trọng số W (từ input vào hidden layer):

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \sum_{i=0}^t \frac{\partial L_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial h_t} \cdot \frac{\partial h_t}{\partial h_i} \cdot \frac{\partial h'_i}{\partial W}$$

Cốt lõi vấn đề của BPTT

Nhìn vào công thức tính đạo hàm của L với U và W ở trên, ta thấy xuất hiện nhân tử:

$$\frac{\partial h_t}{\partial h_i} = \prod_{j=i}^{t-1} \frac{\partial h_{j+1}}{\partial h_j}$$

Đây chính là nguồn cơn của hiện tượng **Vanishing Gradient** (Triệt tiêu đạo hàm) hoặc **Exploding Gradient** (Nổ đạo hàm) khi chuỗi T lớn.

Chứng minh Toán học: Phân rã Gradient trong RNN

Xét thành phần cốt lõi trong BPTT là đạo hàm trạng thái qua nhiều bước thời gian: $\frac{\partial h_T}{\partial h_k} = \prod_{j=k}^{T-1} \frac{\partial h_{j+1}}{\partial h_j}$.

Do $h_{j+1} = \tanh(U \cdot h_j + W \cdot x_{j+1})$, ma trận Jacobian tại mỗi bước là: $\frac{\partial h_{j+1}}{\partial h_j} = \text{diag}(1 - h_{j+1}^2) \cdot U$.

Đánh giá Chuẩn Ma trận (Matrix Norm)

Áp dụng tính chất $\|A \cdot B\| \leq \|A\| \cdot \|B\|$, ta lấy chuẩn 2 về:

$$\left\| \frac{\partial h_{j+1}}{\partial h_j} \right\| \leq \|\text{diag}(1 - h_{j+1}^2)\| \cdot \|U\| \leq 1 \cdot \|U\| = \|U\|$$

(Vì đạo hàm của tanh lớn nhất bằng 1 nên $\|\text{diag}(1 - h_{j+1}^2)\| \leq 1$).

Cơ chế Vanishing Gradient

Nếu trị riêng lớn nhất của U nhỏ hơn 1 (tức $\|U\| = \gamma < 1$): $\left\| \frac{\partial h_T}{\partial h_k} \right\| \leq \gamma^{T-k} \xrightarrow{T-k \rightarrow \infty} 0$.

⇒ Tín hiệu gradient suy giảm theo hàm mũ về 0. Trọng số mất hoàn toàn khả năng cập nhật theo các lỗi từ quá khứ xa.

Hiện tượng Nổ Gradient (Exploding Gradient)

Cơ chế Toán học (Exploding Gradient)

Ngược lại với triết tiêu, nếu trị riêng lớn nhất của U lớn hơn 1 ($\|U\| = \gamma > 1$) và phần lớn nơ-ron hoạt động ở vùng tuyến tính của tanh (nơi đạo hàm xấp xỉ 1):

$$\left\| \frac{\partial h_T}{\partial h_k} \right\| \approx \gamma^{T-k} \xrightarrow{T-k \rightarrow \infty} \infty$$

Hậu quả Hình học ("Vách đá" Hàm Loss)

Sự nhân dồn liên tục của U qua T bước thời gian tạo ra những "vách đá" (cliffs) cực dốc trong không gian tham số.

- Khi mô hình tiến gần vách đá, đạo hàm bùng nổ theo hàm mũ.
- Gradient khổng lồ gây ra bước cập nhật (Overshooting) đẩy văng mô hình khỏi vùng hội tụ, sinh ra lỗi NaN hoặc Inf.

Giải pháp khắc phục: Gradient Norm Clipping

Để ngăn overshooting, ta hãm phanh bước nhảy. Đặt vector $g = \nabla L = \left[\frac{\partial L}{\partial W}, \frac{\partial L}{\partial U}, \frac{\partial L}{\partial V} \right]$ là vector gradient chứa đạo hàm của tất cả các trọng số trong mô hình.

Cơ chế Cắt tỉa (Clipping)

Nếu độ lớn (norm) của vector g vượt ngưỡng θ , ta co giãn toàn bộ vector:

$$g_{\text{clipped}} = \begin{cases} g & \text{nếu } \|g\| \leq \theta \\ \theta \cdot \frac{g}{\|g\|} & \text{nếu } \|g\| > \theta \end{cases}$$

Tại sao dùng Norm Clipping thay vì Value Clipping?

- **Value Clipping (Cắt từng phần tử):** Ép lẻ từng $g_i \in [-\theta, \theta]$. Việc này **làm bóp méo hướng** của gradient, khiến mô hình cập nhật sai hướng.
- **Norm Clipping (Co giãn vector):** Phép nhân vô hướng với tỷ lệ $\frac{\theta}{\|g\|}$ giúp vector mới tuy ngắn lại nhưng **bảo toàn tuyệt đối hướng đi nguyên thủy** trong không gian đa chiều.

**Lưu ý:* Clipping chỉ chống Nổ đạo hàm, hoàn toàn vô dụng với Triệt tiêu đạo hàm.

Từ RNN đến BiRNN: Mở rộng theo hai hướng

Điểm khác so với RNN một chiều

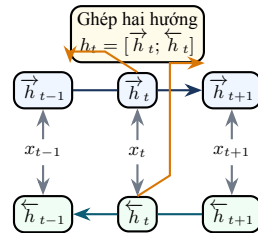
RNN thường chỉ truyền thông tin thuận: $\vec{h}_t = f(x_t, \vec{h}_{t-1})$.

BiRNN thêm nhánh ngược: $\overleftarrow{h}_t = f(x_t, \overleftarrow{h}_{t+1})$, sau đó ghép hai trạng thái:

$$h_t = [\vec{h}_t; \overleftarrow{h}_t]$$

Lợi thế chính

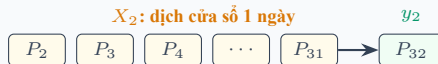
Mỗi vị trí nhận ngữ cảnh từ cả đầu và cuối của **cửa sổ đầu vào**; biểu diễn giàu thông tin hơn RNN một chiều.



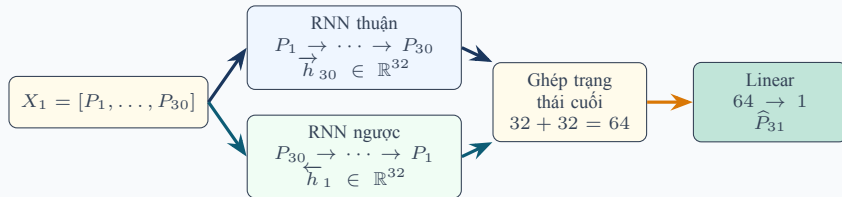
Hai hướng chỉ hoạt động trong cửa sổ dữ liệu đã biết.

BiRNN với giá vàng: Windowing và dự đoán ngày kế tiếp

1. Windowing: tạo các cặp đầu vào–nhận



2. Một cửa sổ được đọc theo hai hướng



BiRNN hồi quy: từ tensor đến giá dự đoán

Code PyTorch cốt lõi

```
import torch
import torch.nn as nn

birnn = nn.RNN(1, 32, batch_first=True,
               bidirectional=True)
head = nn.Linear(2 * 32, 1)

X_batch = torch.tensor(X[:32], dtype=torch.float32) # [32,
              30, 1]
_, h = birnn(X_batch) # [2,
              32, 32]
h = torch.cat([h[0], h[1]], dim=1) # [32,
              64]
y_hat = head(h) # [32, 1]
```

Đọc các kích thước

[32, 30, 1]: 32 cửa sổ, 30 ngày, 1 giá trị/ngày.

[2, 32, 32]: 2 hướng, 32 mẫu, 32 trạng thái.

Ghép hai hướng: $32 + 32 = 64$.

Đầu ra [32, 1]: mỗi cửa sổ cho một giá dự đoán.

Hồi quy giá vàng

Đầu ra là một số thực $\hat{P}_{t+1}$, nên thường dùng MSE hoặc MAE làm loss.

Điều kiện không lộ dữ liệu tương lai

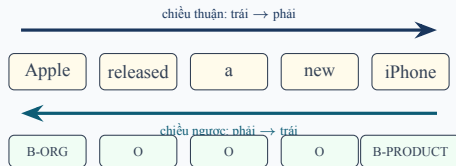
BiRNN chỉ đọc cửa sổ $[P_{t-29}, \dots, P_t]$; giá mục tiêu P_{t+1} chỉ dùng để tính loss, không đưa vào X_batch.

BiRNN trong gán nhãn chuỗi: khai thác ngữ cảnh hai phía

Bài toán phù hợp

BiRNN phù hợp với các nhiệm vụ xử lý **toàn bộ chuỗi đã có sẵn**, trong đó mỗi token cần một nhãn riêng và nhãn đó phụ thuộc cả phần bên trái lẫn bên phải. Ví dụ: nhận dạng thực thể (NER) hoặc gán nhãn từ loại (POS tagging).

Ví dụ: nhận dạng thực thể (NER)



Với token “Apple”, ngữ cảnh bên phải “released a new iPhone” giúp mô hình hiểu đây là một **tổ chức**, không phải một loại quả.

BiRNN tạo nhãn như thế nào?

$$\vec{h}_t = f(x_t, \vec{h}_{t-1})$$

$$\overleftarrow{h}_t = f(x_t, \overleftarrow{h}_{t+1})$$

$$h_t = [\vec{h}_t; \overleftarrow{h}_t]$$

$$\hat{y}_t = \text{softmax}(Wh_t + b)$$

Mỗi vị trí (t) nhận một vector chứa thông tin từ **hai phía**, rồi dự đoán một nhãn riêng cho token đó.

Điểm mạnh và giới hạn

Mạnh: gán nhãn token, NER, POS tagging và phân đoạn chuỗi khi toàn bộ chuỗi đã có.

Giới hạn: không phù hợp với dự báo online/causal, vì hướng ngược cần dữ liệu ở phía tương lai.